

Putting Portfolio Wisdom into Neural Networks

Ton Vossen

github.com/KoperSloper
ton.vossen@outlook.com

Abstract

Most people treat neural networks as black boxes, but usually, we can do better by nudging them with a bit of domain wisdom. This project introduces *Deep Risk Parity*: a simple way to bake a classical portfolio solver directly into a neural net. Instead of letting the model "guess" weights randomly, I used the Implicit Function Theorem to create a differentiable bridge between a Newton-Raphson solver and the network's loss function. It turns out that giving the model a little bit of structural help goes a long way. In my tests, this hybrid approach easily beat basic benchmarks like equal-weighting and nominal risk parity, suggesting that you can actually teach an old dog (risk parity) new tricks (backpropagation).

1. Introduction

After taking the [Matrix Calculus](#) course from MIT, I had a bit of a "Matrix" moment. I realized just how much of the world around us is actually differentiable (Figure 1).

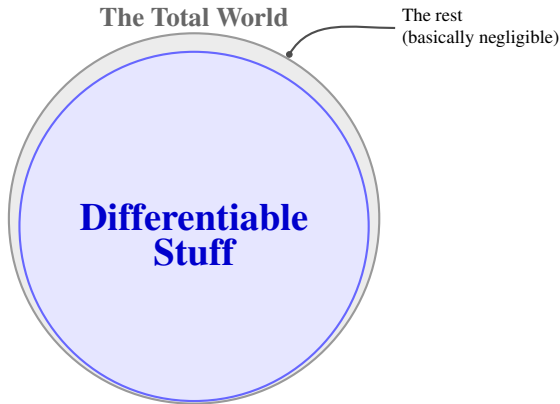


Figure 1: A representation of my post-course worldview.

The logic is simple: if something is differentiable $\implies$ we can put it in a neural network. And as a general rule of thumb, if you can place something in a neural network, you probably should ;).

Since I'm interested in finance, I wanted to see how to incorporate "inductive biases" (or less fancy: nudging your model in a direction you already know is smart) into flexible architectures. Similar to the arguments in [1], structural nudges spare the model from having to learn the laws of physics (or finance) from scratch.

Risk parity is a classic strategy popularized by Ray Dalio's Bridgewater Associates. It delivers great out-of-sample diversification by asking how much *risk* you allocate to a stock rather than just capital. While it looks like a hard-coded optimization step, it's actually differentiable. The code is available on [Github](#).

1.1. What is Risk Parity?

If total risk is $\sigma(y) = \sqrt{y^\top \Sigma y}$, the marginal risk contribution (MRC) of an asset i is $\frac{(\Sigma y)_i}{\sqrt{y^\top \Sigma y}}$ (see A.1 for a derivation). The risk contribution (RC) is then simply the MRC times its weight y_i :

$$RC_i = \frac{y_i (\Sigma y)_i}{\sqrt{y^\top \Sigma y}},$$

which decomposes total risk as $\sigma(y) = \sum RC_i$. The goal is to find weights y that match a budget vector b ($1^\top b = 1$) (see A.2 for a derivation):

$$\text{diag}(y) \Sigma y = b (y^\top \Sigma y)$$

Defining $x = y / \sqrt{y^\top \Sigma y}$ lets us rewrite this as the system $\Sigma x = b/x$. We find the solution x by finding the root of [3]:

$$F(x) = \Sigma x - b/x = 0$$

which I do via the multivariate Newton Raphson method.

2. Differentiability

A few weeks ago, I would have looked at a root-finding process like $F(x)$ and assumed it was a "black box" that gradients couldn't touch. But once you start looking at the world through the lens of matrix calculus, you realize that almost any well-behaved system can be opened up.

To find the Jacobian $\frac{\partial F}{\partial x}$, we take the total differential of $F(x) = \Sigma x - b \circ x^{-1}$:

$$dF = \Sigma dx - b \circ d(x^{-1})$$

$$dF = \Sigma dx - b \circ (-x^{-2} \circ dx)$$

$$dF = (\Sigma + \text{diag}(b/x^2)) dx$$

This gives us the exact Jacobian required for our Newton-Raphson updates:

$$\implies \frac{\partial F}{\partial x} = \Sigma + \text{diag}(b/x^2)$$

While we are at it, we can continue with this momentum to find the derivative with respect to the risk budgets b :

$$\begin{aligned} dF &= -d(b) \circ x^{-1} = -\text{diag}(1/x)db \\ \implies \frac{\partial F}{\partial b} &= -\text{diag}(1/x) \end{aligned}$$

2.1. Vector-Jacobian Products

In modern frameworks like `jax`, we don't actually want to compute or store massive Jacobian matrices. We just need to handle the vector-jacobian product (VJP). When $F(x)$ has converged, $F(x) = 0$. The Implicit Function Theorem (IFT) tells us:

$$\frac{\partial F}{\partial x} \frac{dx}{db} + \frac{\partial F}{\partial b} = 0$$

We can solve for $\frac{dx}{db}$ and plug it into our loss L :

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial x} \left[- \left(\frac{\partial F}{\partial x} \right)^{-1} \frac{\partial F}{\partial b} \right]$$

To do this efficiently, I define an intermediate vector u :

$$u^\top = -\frac{\partial L}{\partial x} \left(\frac{\partial F}{\partial x} \right)^{-1} \implies \left(\frac{\partial F}{\partial x} \right)^\top u = -v_x$$

Once a linear solver hands us u , the final gradient with respect to b collapses into a beautifully simple element-wise division:

$$\nabla_b L = \left(\frac{\partial F}{\partial b} \right)^\top u = -\frac{u}{x}$$

3. Implementation

I simulated four asset classes (large cap, mid cap, small cap, bonds) driven by a dividend yield state variable d_t using parameters from [2]. Log returns r_t and yield d_{t+1} follow linear/AR(1) paths:

$$r_t = a_{1:4} + b_{1:4}d_t + \epsilon_{1:4,t}$$

$$d_{t+1} = a_5 + b_5d_t + \epsilon_{5,t}$$

A "sample" is a 48-month trajectory. I aim to maximize expected CRRA utility: $\mathbb{E}[U(W)] = \mathbb{E}\left[\frac{W^{1-\gamma}}{1-\gamma}\right]$. At each step, an MLP observes d_t and Time-to-Maturity (TTM) to output risk budgets b , which our differentiable solver converts to weights y .

Furthermore, I implemented a "free pool" mechanism to enforce a minimum risk contribution for each asset. Setting `min.budget` to 0.05 ensures that every asset accounts for at least 5% of the total portfolio risk. The `free.pool` represents the remaining risk budget after these floors are satisfied: $1.0 - (\text{K.assets} * \text{min.budget})$. The final risk budget is then calculated as `final_b = min.budget + (raw_b * free.pool)`, where `raw_b` is the normalized softmax output of the network. This ensures the model maintains a baseline level of diversification even when the MLP strongly favors specific assets.

3.1. Results

Deep Risk Parity achieves the highest Certainty Equivalent (CE) return, outperforming both 1/N and nominal risk parity benchmarks.

Strategy	CE (SE)	Max DD	Sortino
Deep Risk Parity	0.0459 (0.0007)	0.1527	1.240
Nominal Risk Parity	0.0377 (0.0005)	0.1198	1.202
Equal Weight (1/N)	0.0376 (0.0011)	0.2437	0.9680

Table 1: Out-of-sample performance metrics.

The model essentially "learned" to tilt toward small caps when yields are high while using the Risk Parity layer to keep the overall risk profile in check (see Figure 2).

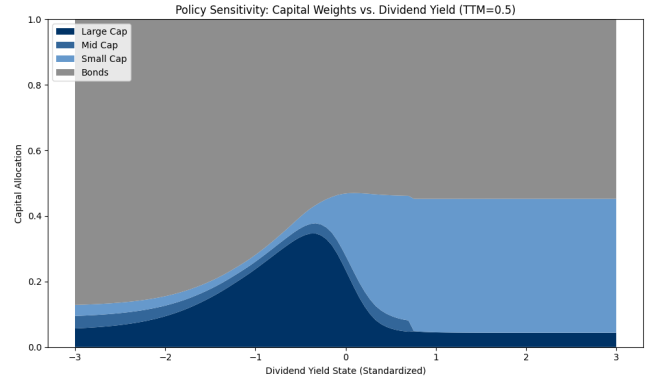


Figure 2: Capital allocation as a function of dividend yield.

4. Conclusion

This project was a fun exploration of how we can combine classical financial solvers with modern deep learning. The key takeaway is that we don't have to choose between a hard-coded "expert" model and a pure "black-box" neural network. The two can be mixed.

This kind of "differentiable programming" is powerful because it is modular. You can swap in different solvers, different loss functions, or different state variables, and the gradients just handle the rest.

There are plenty of ways to break this further. For now (for me), it is just cool to see that you can actually backpropagate through a Newton-Raphson solver and get something that looks like financial wisdom out the other side.

References

- [1] Panagiotis D. Grontas, Antonio Terpin, Raffaello D'Andrea, Efe C. Balta, and John Lygeros. Pinet: Optimizing hard-constrained neural networks with orthogonal projection layers. *arXiv preprint arXiv:2508.10480v1*, 2025.

- [2] Anthony W Lynch. Portfolio choice and equity characteristics: Characterizing the hedging demands induced by return predictability. *Journal of Financial Economics*, 62(1):67–130, 2001.
- [3] Daniel P. Palomar. *Portfolio Optimization: Theory and Application*. Cambridge University Press, 2025.

A. Derivations

A.1. Derivation of Marginal Risk Contribution

Define portfolio variance as $u = y^\top \Sigma y$.

$$du = dy^\top \Sigma y + y^\top \Sigma dy = 2y^\top \Sigma dy$$

Since $\sigma = \sqrt{u}$, its differential is:

$$\begin{aligned} d\sigma &= \frac{1}{2\sqrt{u}} du = \frac{1}{2\sqrt{y^\top \Sigma y}} (2y^\top \Sigma dy) \\ \implies \nabla_y \sigma &= \frac{\Sigma y}{\sqrt{y^\top \Sigma y}} \end{aligned}$$

A.2. Derivation of the Risk Parity Vector Formulation

To achieve a specific risk allocation b , we require the risk contribution of each asset to match its allocated budget relative to the total portfolio risk:

$$\begin{aligned} y_i \cdot \frac{(\Sigma y)_i}{\sigma(y)} &= b_i \cdot \sigma(y) \\ y_i \cdot (\Sigma y)_i &= b_i \cdot \sigma^2(y) \\ y_i \cdot (\Sigma y)_i &= b_i (y^\top \Sigma y) \\ \implies \text{diag}(y) \Sigma y &= b (y^\top \Sigma y) \end{aligned}$$